Expressions and stored values

Track the result of the expression separately from the object it changes.

One cell across successive statements

Start	a = x++	b = ++x
S0 x : int 2	S0 x : int 3	S0 x : int 4
no result assigned	a receives 2	b receives 4
same x cell	same x cell	same x cell
The object changes; its identity stays S0.	The object changes; its identity stays S0.	The object changes; its identity stays S0.

C++17 / FOCUSED FRAGMENT

```
int x=2;
int a=x++;    // a=2, x=3
int b=++x;    // b=4, x=4
int c=x+5;    // c=9, x stays 4
```

- Declaration introduces an object; initialization gives its starting value; later assignment writes an existing object. A discarded `x+5;` does not store its result.
- Postfix supplies the old value and still changes x. Prefix supplies the updated value. Carry the final x into the next statement.
- `double d=5/2;` stores 2.0: integer division happens first. `double(5)/2` gives 2.5; `double(5/2)` is too late. In-range floating-to-int conversion truncates toward zero: `int(-4.8)` is -4.
- Zero is false; any nonzero number, including a negative one, is true. `=` assigns; `==` compares. `sizeof(char)==1`; other byte sizes need an explicit platform assumption.
- Use `int main()` in a normal hosted program. `std::cin>>x` extracts input; `std::cout<<x` inserts output. `#include <iostream>` provides their declarations. C++ supports procedural and OOP styles.

Watch the distinction. Precedence groups an expression; it does not authorize arbitrary repeated side effects on one scalar. Trace separate statements unless the sequencing is clear.

PREDICT BEFORE CHECKING

Starting fresh with `int x=5; int a=x++; int b=++x;`, what are a, b, x?

Answer: 5, 7, 7. Postfix supplies 5 and leaves 6; prefix then changes 6 to 7.

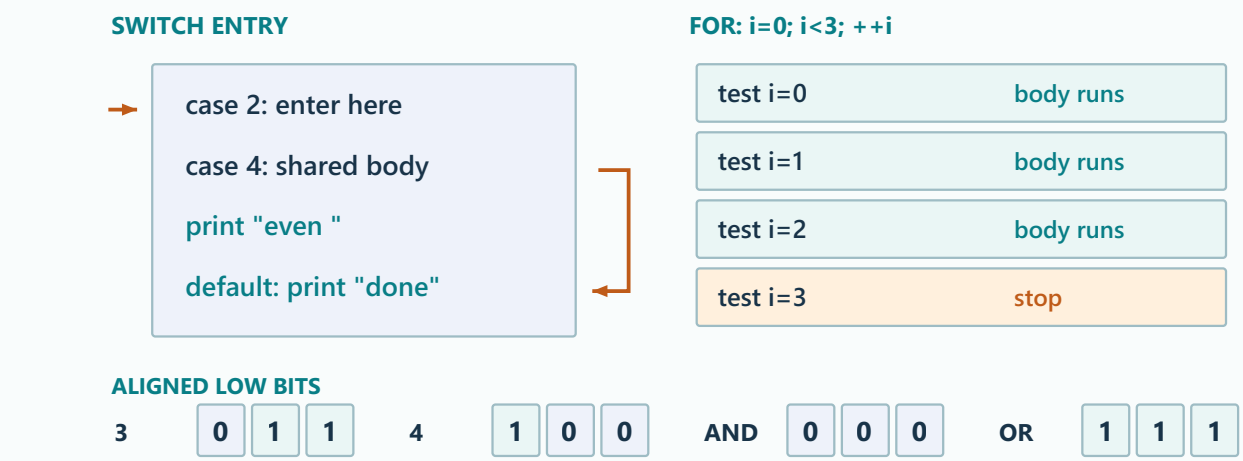
Memory columns show selected objects, not physical byte order. S/A/G/H are symbolic addresses; padding and implementation metadata may be omitted.

Follow the execution path

Find the entry, then follow the statements that actually execute.

Use a path when the question is about execution

Control follows a path



Labels choose entry. Tests decide repetition. Bit positions decide bitwise results.

C++17 / FOCUSED FRAGMENT

```
switch(2) {
    case 2:
    case 4: std::cout<<"even ";
    default: std::cout<<"done";
} // even done: no break before default
```

- `!` reverses truth; `&&` needs both true; `||` needs one true. Built-in AND/OR skip an unnecessary right operand. Grouping precedence: `!`, then `&&`, then `||`.
- Switch cases require distinct integral/enum constant values. `case 4/2:` duplicates `case 2:`. Default is optional; unmatched input without default executes no case body. Fallthrough does not retest labels.
- `while` tests first; `do/while` tests after the body. The for path and its final failed check are drawn above.
- `break` exits the nearest loop/switch. `continue` skips the remaining iteration; a for loop still performs its update. Two counters in one loop are not nested loops. A missing for condition means true.
- Logical `3&&4` is true. The bit lanes show AND 0/OR 7. `18u>>1` is 9; `(~7u)&255u` keeps a low-eight-bit complement 248. Test a valid bit k with `flags & (1u<<k)`.

Watch the distinction. `x%2==0 ? "even" : "odd"` tests the comparison. An even remainder is 0, but the comparison is true. A loop does not create a fresh recursive frame per iteration.

PREDICT BEFORE CHECKING

`int i=3; do{++i;}while(i<5);` How many bodies execute?

Answer: 2. The body changes 3 → 4, then 4 → 5; the second check fails.

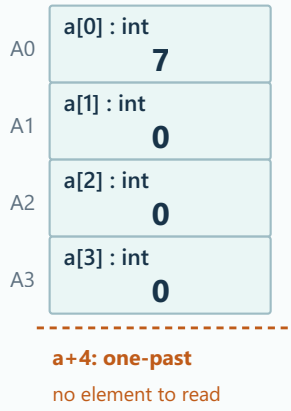
Read only the cells that exist

An index chooses an element; a member name chooses a field; a null character ends a C string.

Columns reveal bounds and stored content

Element column

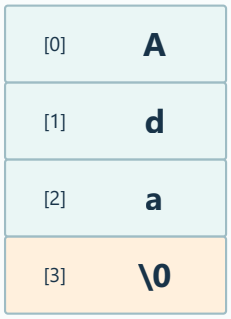
int a[4] = {7}



Cells are contiguous elements, not unrelated variables.

Terminated text

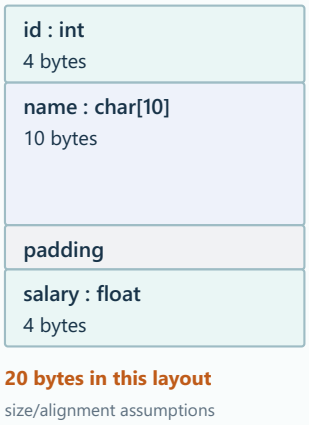
char name[]="Ada"



The terminator occupies storage.

Mixed fields

Employee example



Padding is layout space, not a data member.

C++17 / FOCUSED FRAGMENT

```
int a[4]={7};           // 7,0,0,0
char name[]="Ada";      // A,d,a,'\0'
int grid[2][3]={{1,2,3},{4,5,6}};
// grid[1][2] == 6
```

- Four array elements have indices 0-3. `a+4` is one-past: forming it is allowed; reading `a[4]` is not. Typed `a+i` already advances i elements, not i bytes.
- An ordinary uninitialized local `int a[4];` is not automatically zeroed. Partial brace initialization, as above, initializes the remaining ints to zero.
- A 2D array contains row arrays. `grid[row][col]` selects within a row; nested loops visit 2×3 cells. Preserve row boundaries when using pointers.
- `"Ada"` needs four chars including `\0`, although the visible length is three. `char n[3]="Ada";` is invalid C++; three letters without a terminator are not a C string.
- A struct can contain mixed member types. Padding can enlarge its size. `p->salary` means `(*p).salary`. An array of Employee objects still has one element type: Employee.

Watch the distinction. Exam focus: “array is a constant pointer.” Intended cue: the array name cannot be reseated, while mutable elements can change. Precisely, an array is its own type and often converts to its first-element pointer. `sizeof(a)` measures the complete actual array; a parameter declared `int a[]` adjusts to a pointer, so pass a count.

PREDICT BEFORE CHECKING

Can `name[1]='o';` change the array? Can `name=other;` reseat it?

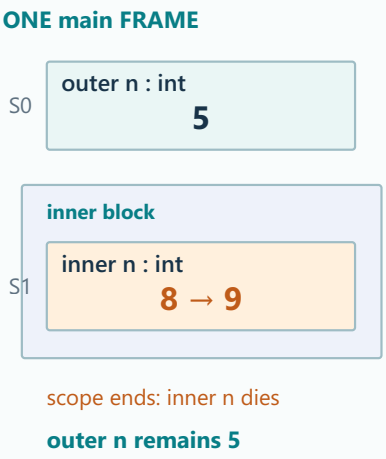
Answer: the element write is valid; array assignment/reseating that way is invalid.

Local name, lasting state

Separate where a name is usable from how long its object remains alive.

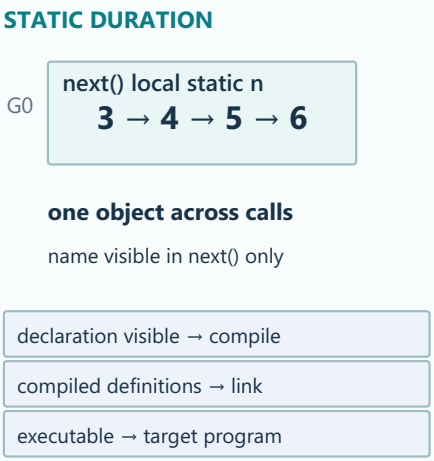
Different names and lifetimes need different cells

Shadowing is a second object



The name repeats; the stored object does not.

Persistence is not visibility



G0 is a symbolic static-storage label.

C++17 / FOCUSED FRAGMENT

```
int next(){ static int n=3; return ++n; }
// successive calls in one run: 4,5,6
int n=5;
{ int n=8; ++n; } // changes the inner object only
// outer n is still 5
```

- An inner declaration can shadow an outer name. It creates a different object; it does not overwrite the outer one.
- An ordinary local initialized to 3 restarts each call; a local static initializes once and persists. Its name stays local; its value is not a disk save. Globals/ statics also exist without new.
- A declaration describes the callable interface; a definition supplies its body. A suitable declaration must precede use: `int sum(int,int);` declares a result and two parameter types.
- `#include "Weapon.h"` exposes declarations. `void Weapon::attack(){...}` defines that member; free `void attack(){...}` is different. Compile the required cpp files and link them.
- No visible declaration: compilation problem. Required compiled definition absent: commonly a linker problem. An object file is compiled code, not a class instance.

Watch the distinction. After `cin>>age`, a leftover newline can make the next getline empty. Consume the old line's remainder before reading the next full line.

PREDICT BEFORE CHECKING

Does a static local inside main become visible in another function?

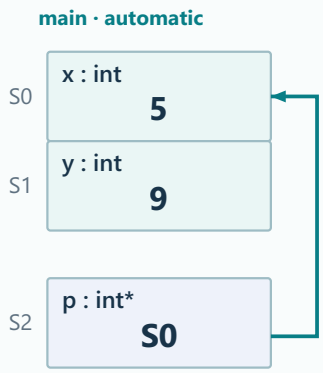
Answer: no. Its name remains local even though its object lasts for the program run.

Write the target, or move the route?

The pointer cell and the pointed-to cell are separate objects.

Keep the cells fixed; watch what changes

1 · Before

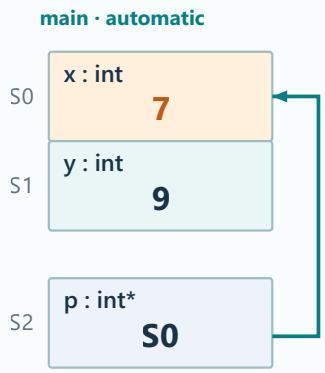


&p is S2

***p is 5**

p stores S0 and reaches x.

2 · *p = 7

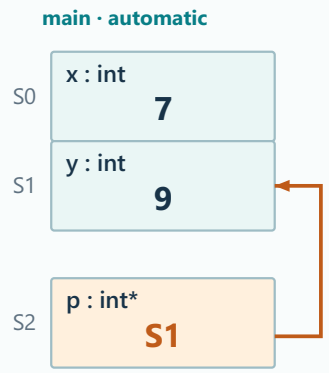


&p is S2

***p is 7**

Only x changes. The route still reaches x.

3 · p = &y



&p is S2

***p is 9**

Only p changes now. x remains 7.

C++17 / FOCUSED FRAGMENT

```
int x=5, y=9;
int* p=&x;
*p=7;        // writes x
p=&y;        // changes p; x stays 7
```

- `&x` is x's address. The value stored in p is a target address. `*p` designates the target object. `&p` is the pointer object's own address.
- `int* q=p;` copies the address into a new pointer cell. Both routes initially reach x. `q=&y` moves q alone; writing `*q` changes q's current target.
- Pointer type governs lawful access and stepping. Equal byte sizes do not make unrelated object types interchangeable through a cast.
- `int* p,q;` declares one pointer and one int. Initialize an absent target with nullptr. Null, uninitialized and dangling pointers have different problems; none is safe to dereference.
- Read arrows from stored addresses to live objects. Physical proximity does not authorize reading a nearby unrelated variable.

Watch the distinction. `*p+5` computes a value; `*(p+1)` reads another element only if it exists. Neither expression assigns a new address to p. Only an operation such as `++p` or `p=&y` changes the route.

PREDICT BEFORE CHECKING

After the shown code, what are x, y, *p? Which address does &p obtain?

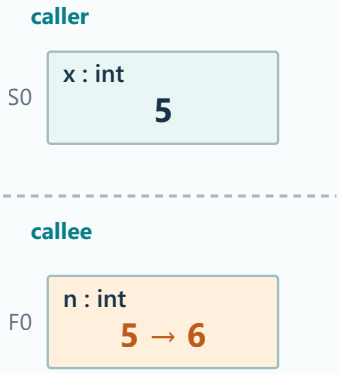
Answer: 7, 9, 9. &p obtains the address of the pointer cell, not x or y.

Which call can change x?

Compare all three calls from a fresh x=5. The parameter declaration explains the difference.

A different parameter creates a different memory story

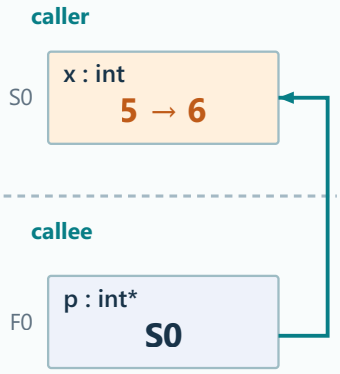
value(x)



two int objects

Increment the new n; caller x stays 5.

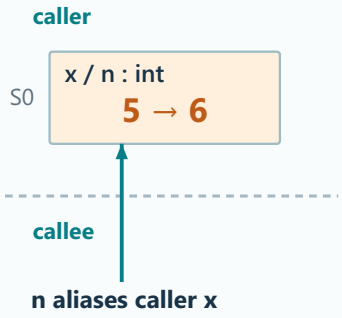
address(&x)



new pointer; old int

Dereference the copied route; caller x changes.

reference(x)



one int, two names

Use another name for the original int.

C++17 / FOCUSED FRAGMENT

```
void value(int n)    { ++n; }
void address(int* p) { ++(*p); }
void reference(int& n){ ++n; }
// value(x); address(&x); reference(x);
```

- Value: picture `int n=x`. A second int receives 5; changing n leaves the caller's int 5.
- Address: picture `int* p=&x`. A new pointer receives an address, so `++*p` changes the original x to 6. Reassigning local p does not reseat the caller's pointer.
- Reference: picture `int& n=x`. There is one int with two names; `++n` changes that int to 6. No separate int is copied for this alias.
- A reference must bind to a suitable live object and cannot reseat. With a valid p to x, `int& r=*p` binds r to x too. References can still dangle if the target dies.
- `int& r=x; r=y;` copies y's value into x; it does not move r onto y. Later changing y alone does not change x. A const reference can provide read-only access without a value copy.

Watch the distinction. "Pass by value" describes the parameter object. A class copied by value may still share a raw-pointer target. It is not a universal guarantee of deep resource copying.

PREDICT BEFORE CHECKING

With x=5, y=7 and r bound to x: `r=y; ++r;` What are x, y?

Answer: 8, 7. The alias stays attached to x throughout both writes.

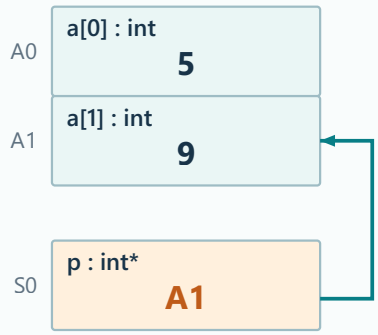
Move a pointer, or increment a value?

Start two independent arrays with the same values, then compare the changed cells.

Same starting values; different object increments

old = *p++

After the expression

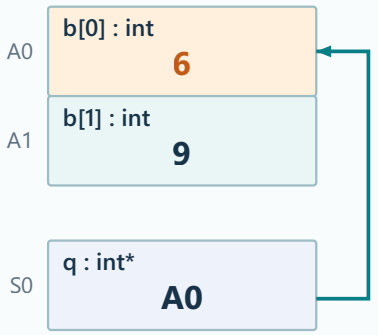


old result = 5

Move the route; keep the old target value.

old2 = (*q)++

After the expression



old result = 5

Write the target; keep the route.

C++17 / FOCUSED FRAGMENT

```
int a[2]={5,9}; int* p=a;
int old=*p++;   // *(p++): old=5, p now points at a[1]
int b[2]={5,9}; int* q=b;
int old2=(*q)++; // old2=5, b[0]=6; q stays at b[0]
```

- In `*p++`, postfix binds more tightly than dereference. It supplies the old pointer for the read, then advances the pointer. Parentheses in `(*p)++` instead select the pointed-to value to increment.
- `p+n`/`p-n` move by elements within one array, including one-past. `p-q` counts elements when both positions belong to the same array or its one-past boundary. `p+q`, pointer multiplication/division and `p-=q` for two pointers are invalid.
- `int** pp=&p`: *pp designates p; **pp designates p's int target. `*pp=&y` reseats p; `**pp=8` writes the current target.
- `int* const p` fixes the pointer; `const int* p` forbids writing the int through p; `const int* const p` restricts both. A mutable x may still change through another access path.
- `void*` holds a generic object address but cannot be directly dereferenced or advanced in standard C++. Convert back to a suitable actual type; a cast does not create that type's object.

Watch the distinction. One-past is not another element. A lone int allows a one-past pointer to form, but reading there is invalid. Check bounds before predicting a value.

PREDICT BEFORE CHECKING

For `int a[3];`, what is `(a+2)-a`? Is `(a+2)+a` valid?

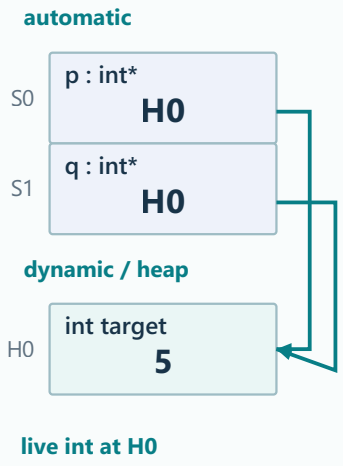
Answer: the distance is 2 elements. Adding the two pointers is invalid.

The pointer survives; the target may not

Draw automatic storage and dynamic storage as separate columns.

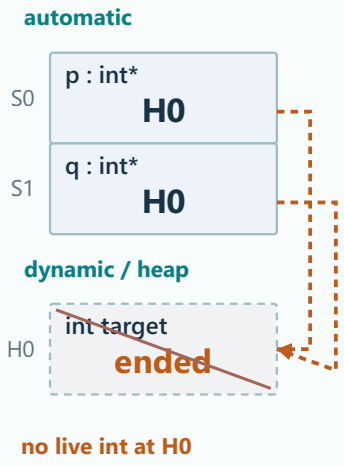
End the target lifetime; do not erase the aliases

1 · Allocate and alias



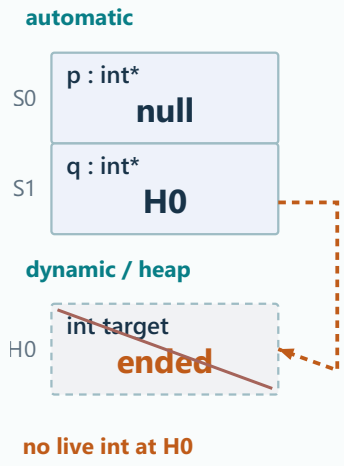
Two pointer cells reach one live int 5.

2 · delete p



The target dies; both old routes dangle.

3 · p = nullptr



Only p is cleared. q still dangles.

C++17 / FOCUSED FRAGMENT

```
int* p=new int(5);
int* q=p;              // two routes, one target
delete p;             // ends that target's lifetime
p=nullptr;            // clears p only; q is dangling
```

- `new int(5)` initializes to 5; `new int{}` initializes to zero; plain `new int` does not zero-initialize that scalar. Equal values do not imply shared objects: a local float 5.5 and a new float 5.5 are separate.
- A local pointer may have automatic storage while its target is dynamic. Leaving the pointer's scope does not automatically delete its raw-pointer target.
- Match `new T` with delete and `new T[n]` with delete[]. A runtime count is supported by `new int[n]`; a variable-length built-in automatic array is not standard C++17.
- Losing the last owning address can leak storage. Deleting the same target twice is invalid. Do not delete a pointer to an ordinary automatic local.
- Ordinary throwing new reports allocation failure by exception; a nothrow allocation can instead return nullptr. For records, `p->field` accesses the pointed-to object's member while it is alive.

Watch the distinction. delete does not mean “set every pointer to null.” After deletion, the diagram crosses out the target's live state; q still stores the old route and must not be dereferenced.

PREDICT BEFORE CHECKING

After the code, did clearing p also clear q? May `*q` still be read?

Answer: no automatic clearing occurred. q is dangling, so do not read through it.

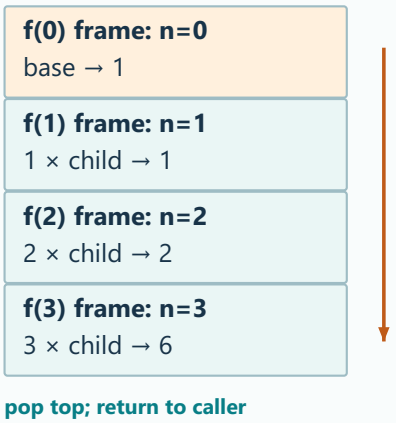
Keep each waiting call on the stack

The base returns one answer; waiting callers still have their own work to finish.

Call-specific data and executable code are different

The newest frame returns first

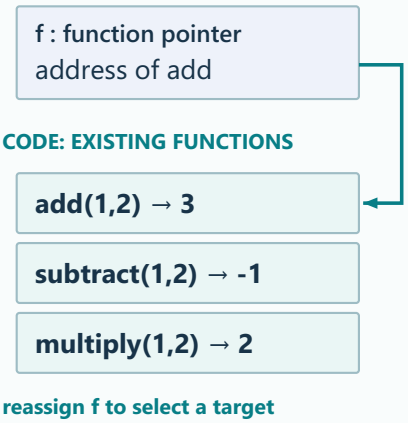
DEEPEST ACTIVE STACK



Each waiting frame keeps its own n and pending work.

A callback chooses code

AUTOMATIC POINTER



The same call shape can select a different function.

C++17 / FOCUSED FRAGMENT

```
int factorial(int n){ // teaching domain: 0<=n<=6
    if(n==0) return 1;
    return n*factorial(n-1);
}
// factorial(3): returns 6 using four factorial calls
```

- Write the input domain, base case and progress step. `0!` is 1. Stopping only at 1 needs zero handled separately.
- Each frame keeps its own n and pending multiplication. The newest frame is on top; returns move down the shown stack: 1, 1, 2, 6. Outer n values are preserved.
- `power(b,p)` for nonnegative p returns 1 when p=0, otherwise `b*power(b,p-1)`. The base b stays fixed; the exponent counts remaining factors. power(2, 3) unwinds 1, 2, 4, 8.
- A negative exponent moves away from zero under this decrementing recurrence. Even terminating inputs can overflow the result type. Work after a recursive call makes this factorial form non-tail-recursive.
- A function pointer selects behavior, not a copy of function code: `int (*f)(int,int)=add;`. After compatible reassignment to subtract or multiply, the same `f(1,2)` gives 3, -1, 2. Callbacks pass such behavior to another function.

Watch the distinction. `int (*f)(int,int)` is a function-pointer declaration; `int* f(int,int)` declares a function returning a pointer. Parentheses change the declaration's meaning.

PREDICT BEFORE CHECKING

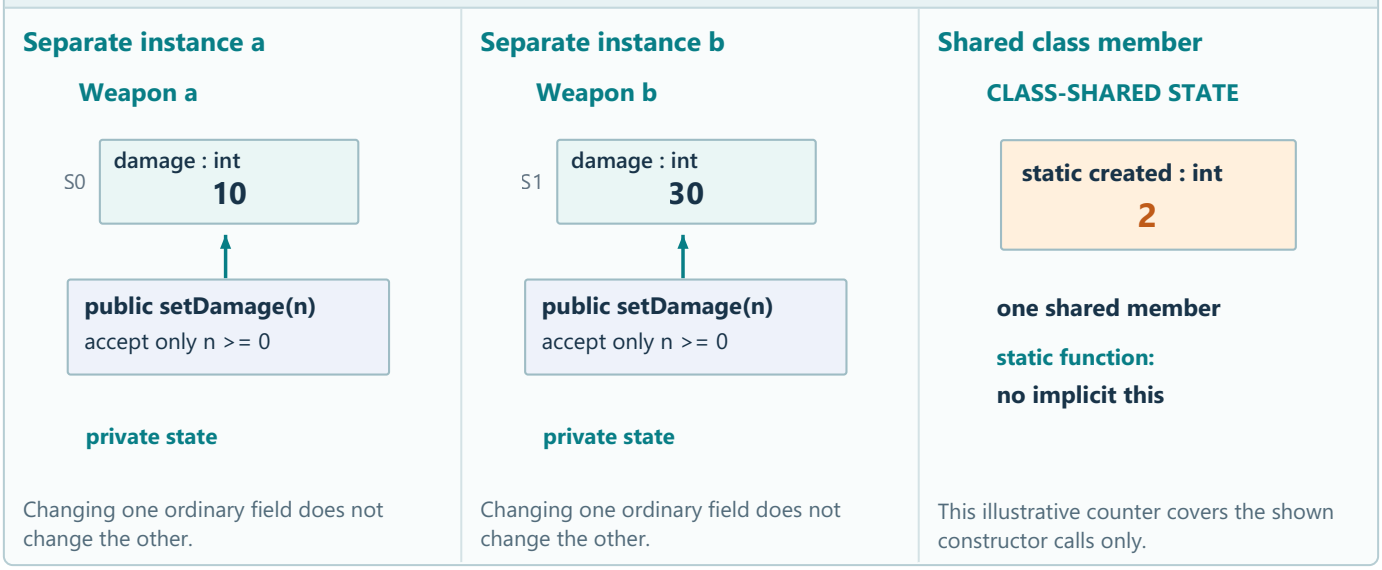
Does factorial(3) use three calls because the input is 3?

Answer: no. With the shown zero base, calls are f(3), f(2), f(1), f(0): four.

Per-object state and shared class state

Two objects have separate ordinary members, even when the type has shared static state.

Two damage cells; one shared counter



C++17 / FOCUSED FRAGMENT

```
class Weapon {
    int damage=0;
    inline static int created=0; // C++17, shared
public:
    Weapon(){ ++created; }
    void setDamage(int n){ if(n>=0) damage=n; }
    int getDamage() const { return damage; }
}; // required semicolon
// Weapon a,b; a.setDamage(10); b.setDamage(30);
```

- Class defines a type; object means instance. `Weapon w;` can be automatic; `new Weapon` is dynamic. Classes do not force heap allocation.
- Encapsulation controls state changes through operations. This setter rejects negative damage. A getter need not have a public setter.
- Abstraction exposes a useful interface and relevant model. Inheritance means is-a; polymorphism allows different implementations through a common interface.
- Class defaults to private member/base access; struct defaults to public. Both support C++ class features, including methods and inheritance.
- Private: class/friends. Protected: also qualifying derived contexts. Public: accessible interface. Each access label continues until the next label/end.
- Static data is shared; ordinary fields are per object. A static function has no implicit this. `Weapon::total()` can be static; an ordinary count needs an instance.

Watch the distinction. The shown counter counts its implemented constructor path, not automatically every copy or live object. A local static is distinct from an ordinary instance field.

PREDICT BEFORE CHECKING

If damage is private, can main write `w.damage=-1` directly?

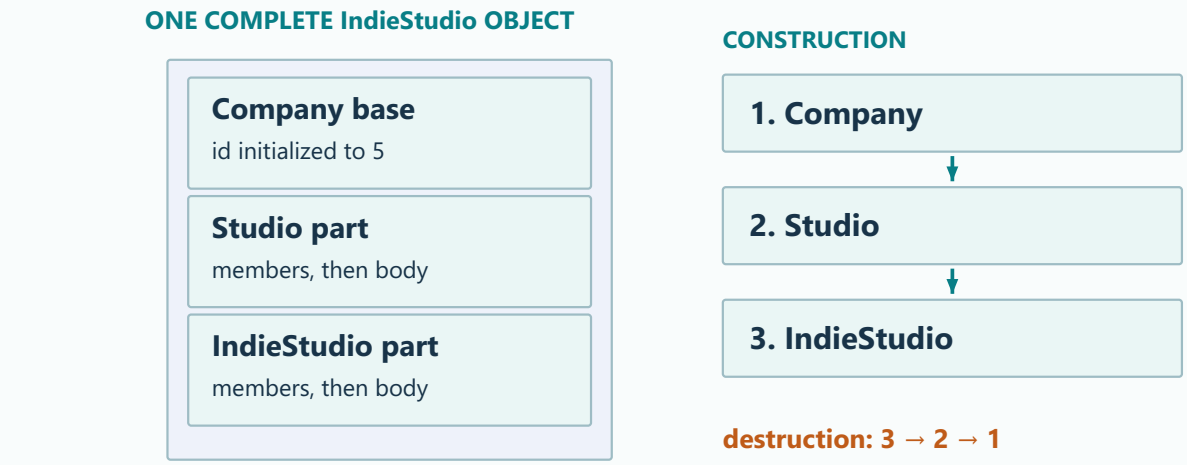
Answer: no. main may request an allowed public operation; its body controls the change.

Initialize the base before the body

Construct the whole object in dependency order; destroy it in reverse.

The foundation exists before the derived body runs

Nested state, ordered lifetime



A base subobject is not another independently allocated heap object.

C++17 / FOCUSED FRAGMENT

```
struct Company { int id;
    explicit Company(int n):id(n){} };
struct Studio : Company {
    Studio() : Company(5) {} // initialize the base first
};
struct IndieStudio : Studio {};
```

- Constructor: class name, no return type. Default means callable without arguments. Declaring an ordinary constructor can suppress the implicit default one. Different argument values do not create overloads.
- Initialize bases, then members in declaration order, then the body. Initializer-list spelling cannot reverse member order.
- `:Company(5)` initializes the base. `Company(5);` in the body instead creates a separate temporary. A default argument is another option.
- `this` points to the current object; `this->id=id` distinguishes member and parameter. Reference/const members require initialization.
- Destructor: `~ClassName()`, no return type or ordinary parameters. Normal local/array destruction reverses construction. Raw dynamic objects need matched release.
- Deleting a derived object through an owning base pointer normally requires a virtual base destructor. Ordinary method calls do not automatically chain base bodies.

Watch the distinction. The base regions belong to one complete object. This is a nesting/lifetime model, not exact byte order or separate heap allocations.

PREDICT BEFORE CHECKING

For Company→Studio→IndieStudio, which constructor finishes first? Which destructor body runs first normally?

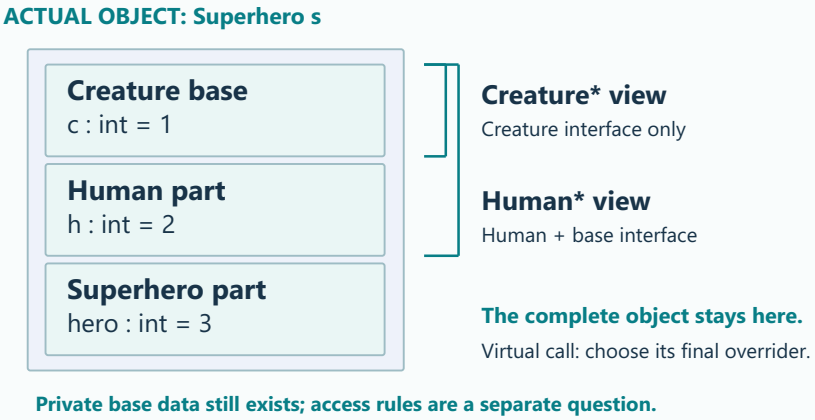
Answer: Company constructor; IndieStudio destructor body.

Same object, different visible interface

A base pointer changes the view; it does not turn a Superhero into a Creature object.

Object contents and method selection are different decisions

A view does not erase cells



For each call: identify static view, then check whether that declaration is virtual.

C++17 / FOCUSED FRAGMENT

```
Superhero s; Creature* p=&s; p->walk();
```

walk first virtual at	Creature* → Superhero	Human* → Superhero
nowhere	Creature::walk	Human::walk
Creature	Superhero::walk	Superhero::walk
Human only	Creature::walk	Superhero::walk

- Public, accessible, unambiguous inheritance allows upcasting. The reverse implicit conversion is invalid: base objects lack derived state.
- Static type selects the interface; a matching virtual call selects the actual object's final overrider. Private base data still exists.
- Without its own override, Superhero may inherit Human's. `override` checks a matching virtual relationship.
- A base-pointer array preserves distinct objects for one command loop. A separate base-value copy can slice; this view does not.
- `virtual void attack()=0;` is pure. Abstract types allow declarations and pointers/references, but not instances. An unimplemented pure operation keeps a child abstract.

Watch the distinction. Use the table with matching walk declarations. Pure virtual blocks instantiation, not declaration of the abstract type.

PREDICT BEFORE CHECKING

Virtual starts only at Human. A Creature* points to Superhero. Which walk runs?

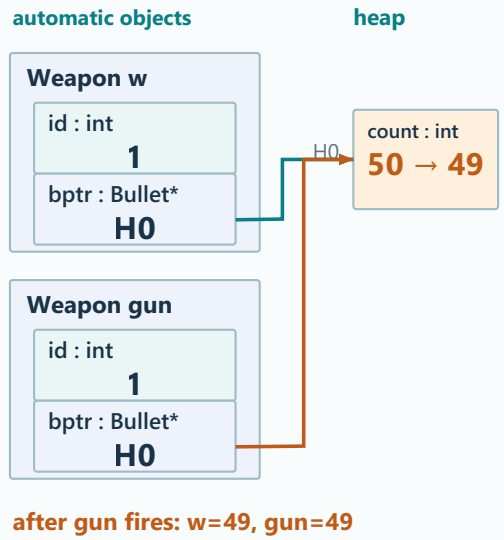
Answer: Creature's nonvirtual walk. The Human declaration does not change that call.

Copy the address, or copy the target?

Draw the pointer members separately from the resource they reach.

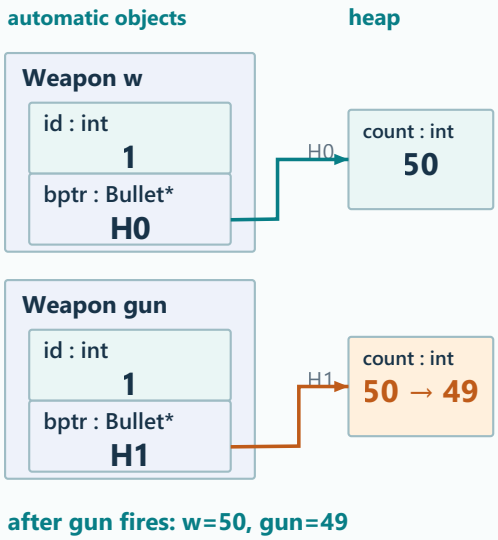
Follow the member arrows before predicting a mutation

Shared pointer target



One Bullet object; both routes reach H0.

Independent target copy



Two Bullet objects; gun changes H1.

C++17 / FOCUSED FRAGMENT

```
// Inside Weapon's copy constructor; other.bptr is live:
this->id = other.id;                    // independent scalar
this->bptr = new Bullet(*other.bptr);    // independent target
```

- Implicit copies are memberwise: ordinary ints become independent; pointer cells copy addresses and can share targets.
- In the drawing, firing gun changes the reached Bullet. Shared target: both see 49. Independent targets: w sees 50, gun sees 49. Scalar id fields remain independent.
- `Weapon gun(w);` or `Weapon gun=w;` at declaration constructs. Later `gun=w;` assigns. One custom copy constructor does not repair assignment/cleanup.
- Owning raw pointers need consistent destructor, copy constructor and copy assignment. Shallow owners may double-delete; value members/RAII simplify ownership.
- `new Bullet(*other.bptr)` uses Bullet's own copy semantics. Dereferencing an uninitialized bptr to copy into it is invalid. A class member follows its own copy behavior.
- Return-by-value may use moves/elision; no universal copy count. A value parameter can still share a raw-pointer target.

Watch the distinction. `gun.id` changes a scalar member. `gun.bptr->count` changes the reached Bullet. Follow the arrow before predicting who observes a write.

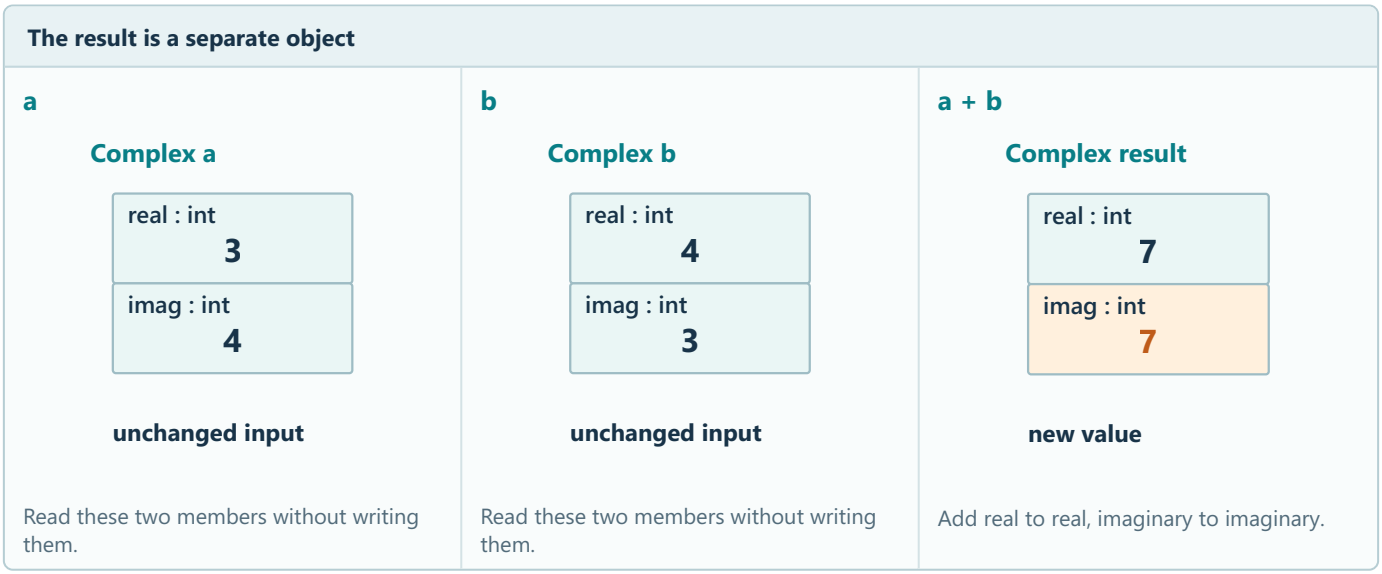
PREDICT BEFORE CHECKING

After shallow memberwise copying, does `gun.bptr=&anotherBullet` automatically reseat w.bptr?

Answer: no. The pointer cells are separate, even when their earlier target was shared.

Same name or symbol, precise meaning

Separate overload selection, virtual overriding and the state changes chosen by an operator body.



C++17 / FOCUSED FRAGMENT

```
Complex operator+(const Complex& rhs) const {
    return Complex(real+rhs.real, imag+rhs.imag);
}
// Member of Complex; constructor takes real,imag.
// (3+4i)+(4+3i) produces 7+7i.
```

- Overloads need distinguishable parameter lists; parameter names and return type alone do not suffice. Different values calling one signature are not overloading.
- Override: a matching inherited virtual operation. Operator overload: an existing symbol's class behavior. Static functions cannot be virtual.
- Nonmutating `a+b` returns a separate result. The member form sees a as `*this`, b as rhs. `const&` avoids an argument copy and restricts mutation.
- Prefix: `Complex& operator++()`. Postfix: `Complex operator++(int)` normally returns the saved old value. The dummy int distinguishes signatures. This exercise increments both components.
- Do not return a reference to a dead local. A value return fits arithmetic; a heap-pointer result changes ownership. Elision/moves can avoid copies.
- No new ** operator or changes to precedence/ associativity/operand count. `::`, `.` and sizeof cannot be overloaded. Defining + does not automatically make Complex printable.

Watch the distinction. Formatting is separate from arithmetic. A negative imaginary component may need a minus sign. An int plus an empty string is not a general C++ string conversion.

PREDICT BEFORE CHECKING

If Complex increments both fields and a=(3, 4), what does `old=a++` leave?

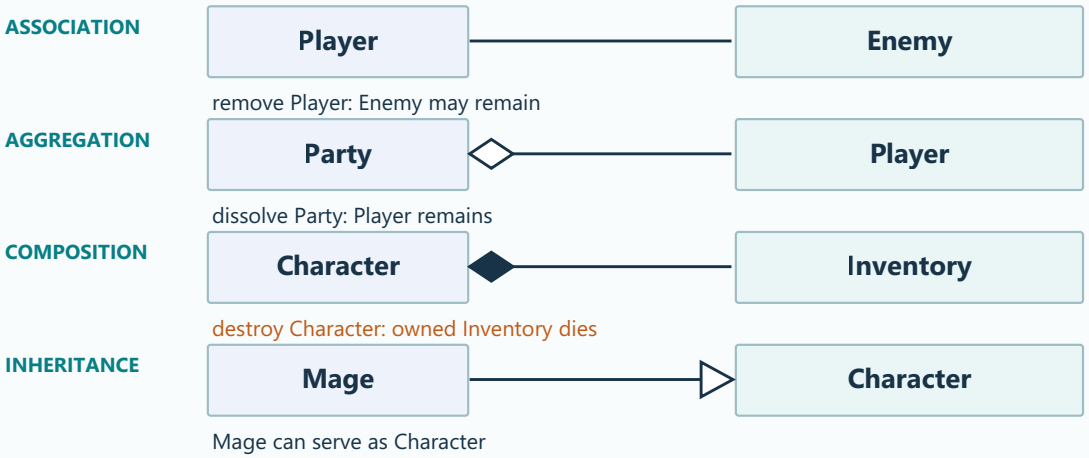
Answer: old=(3, 4), a=(4, 5), under the shown exercise policy.

Let the relationship predict a consequence

Locate the owning whole or the general base before choosing a symbol.

A relationship should predict something observable

Choose the symbol by its consequence



The same nouns could support a different policy. State the policy explicitly.

UML / CLASS COMPARTMENTS

```
Character
-health: int
+attack(): void; +takeDamage(amount: int): void
```

- Association: solid link between independent objects. A pointer does not transfer ownership by itself. Temporary parameter-only use is often a dashed dependency in fuller UML.
- Aggregation: hollow diamond at the grouping whole. In this Party model, Players survive dissolution.
- Composition: filled diamond at the lifetime-owning whole. Character's Inventory dies with it. Referenced items need their own policy.
- Inheritance: hollow triangle points to the base. Mage is a Character; Character has an Inventory is different. Reuse alone does not justify is-a.
- Compartments: name, attributes, operations. Visibility: + public, - private, # protected. Multiplicity: 0..1 optional, * zero-or-more, 1..* one-or-more. Include meaningful methods.
- A nonvirtual Person→Student/Faculty→TA diamond has two Person base subobjects and possible ambiguity. C++ supports multiple inheritance; virtual inheritance is distinct from virtual method dispatch.

Watch the distinction. Choose the lifetime policy before the symbol. A game diagram models behavior; its nouns alone do not prove ownership.

PREDICT BEFORE CHECKING

If a Party dissolves but its Players remain, where does the hollow diamond sit?

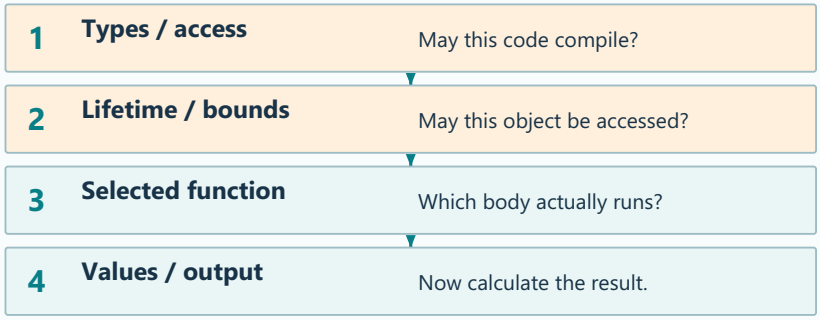
Answer: at Party, the grouping whole. Filled would assert a different ownership model.

Decide validity before calculating a number

Read code in this order: types/access → lifetime/bounds → selected function → values/output.

An ordered reading strategy prevents several MCQ traps

Do not skip the validity gates



A compilation failure or undefined behavior is not an ordinary numeric output.

C++17 / FOCUSED FRAGMENT

```
// Three different categories:
int* p=nullptr;
// *p = 3;       // invalid access if executed
// p + p;        // ill-formed expression
int a[2]={1,2};  // a+2 can form; *(a+2) cannot be read
```

- Prioritize pages 5-8, 12-13 if routes, aliases, bounds or copying are uncertain. Then revisit arrays, lifetime and UML. Each page's question tests a specific distinction.
- For an output question, first allow “does not compile” or “undefined behavior.” An observed accidental value does not establish a language rule.
- For virtual calls, write static type and actual type. For copies, underline declaration versus later assignment. For pointer operations, circle the exact object being changed.
- Keep the flagged array cue and the exact array rule together. It is not a reason to mark mutable array elements const.
- For interface questions, separate access from ownership. Protected is not public; a pointer field alone does not tell who must delete its target.

Watch the distinction. Choose the answer supported by the exact code and stated assumptions, especially element counts, live objects, access labels and where virtual is declared.

PREDICT BEFORE CHECKING

1. `double d=7/2;` stores 3.5 or 3.0? 2. Does `int& r=x; r=y;` rebind r? 3. Does delete clear q when q=p? 4. Can a remaining-pure-virtual child type be declared? 5. Does return type alone overload? 6. Is a base pointer view slicing?

Answers: 1)3.0: integer division first. 2)No: writes x. 3)No: aliases remain dangling. 4)Yes: it remains abstract; instantiation fails. 5)No. 6)No: a separate base-value copy can slice.